

Data Analysis with Python

Lesson 3: Working with Pandas DataFrames

Working with Pandas DataFrames - Study Notes

Key Concepts

1. **DataFrame Structure** – 2 dimensional, labeled axes (rows & columns) that can hold heterogeneous data types.
2. **Loading & Inspecting Data** – `pd.read_csv()`, `head()`, `info()`, `describe()`, and `shape` give quick insight into the dataset.
3. **Filtering & Selecting** – Boolean indexing, `loc[]`, `iloc[]`, and column selection allow precise data extraction.
4. **Grouping & Aggregation** – `groupby()` combined with aggregation functions (`sum`, `mean`, `count`, `agg`) summarizes data by categories.
5. **Data Cleaning Basics** – Handling missing values (`isna()`, `fillna()`, `dropna()`), renaming columns, and type conversion.

Summary

Pandas DataFrames are the backbone of data manipulation in Python. They provide a flexible, tabular structure that can store different data types across columns, making them ideal for real world datasets. The typical workflow starts with loading data from CSV (or other formats) using `pd.read_csv()`, followed by inspecting the data to understand its shape, types, and any missing values. Once the data is understood, filtering and selecting rows or columns isolate the subset of interest. For deeper analysis, `groupby()` partitions the DataFrame into groups based on one or more keys, and aggregation functions compute statistics for each group. Throughout this process, cleaning steps such as filling or dropping missing values, renaming columns, and ensuring correct data types are essential to avoid errors in downstream analysis.

Important Points

- **Reading CSVs**

```
```python
```

```
df = pd.read_csv('data.csv', parse_dates=['date'])
```

```
```
```

`parse_dates` automatically converts columns to datetime objects.

- **Inspecting Data**

- `df.head()` – first 5 rows
- `df.info()` – column types & non null counts
- `df.describe()` – summary statistics for numeric columns
- `df.shape` – (rows, columns)

- **Boolean Indexing**

```

```python
filtered = df[df['age'] > 30]
```



- loc` vs `iloc`
- loc[row_label, column_label] – label based
- iloc[row_index, column_index] – integer based
- Grouping & Aggregation



```

```python
grouped = df.groupby('department').agg(
    total_sales=('sales', 'sum'),
    avg_age=('age', 'mean')
)
```


- Handling Missing Data
- df.isna() – boolean mask
- df.fillna(value) – replace missing values
- df.dropna() – remove rows/columns with missing data
- Data Types


```

```python
df['price'] = df['price'].astype(float)
```



- Renaming Columns



```

```python
df.rename(columns={'old': 'new'}, inplace=True)
```

```


```


```


```

Examples

Example 1 – Load, Clean, and Summarize Sales Data

```

```python
import pandas as pd

sales = pd.read_csv('sales.csv', parse_dates=['order_date'])

```

## 1. Load the CSV

## 2. Inspect

```

print(sales.info())
print(sales.head())

```

## 3. Clean: fill missing prices with 0

```

sales['price'].fillna(0, inplace=True)

```

## 4. Filter: only orders from 2023

```
sales_2023 = sales[sales['order_date'].dt.year == 2023]
```

## 5. Group by product and calculate totals

```
summary = sales_2023.groupby('product_id').agg(
 total_units=('quantity', 'sum'),
 total_revenue=('price', 'sum')
)
.reset_index()
print(summary.head())

```

\*Explanation:\*

- ``parse_dates`` turns ``order_date`` into datetime.
- ``fillna(0)`` ensures no missing prices affect revenue calculations.
- The filter keeps only 2023 orders.
- ``groupby`` aggregates by ``product_id``, summing units sold and revenue.

---

### Example 2 – Pivot Table for Monthly Sales by Region

```
```python
```

Assume 'sales' DataFrame from Example 1

Create a month column

```
sales['month'] = sales['order_date'].dt.to_period('M')
```

Pivot: rows=month, columns=region, values=total revenue

```
pivot = sales.pivot_table(  
    index='month',  
    columns='region',  
    values='price',  
    aggfunc='sum',  
    fill_value=0  
)  
print(pivot)  
---
```

Explanation:

- ``to_period('M')`` converts dates to month periods.
- ``pivot_table`` reshapes data, summing revenue per month region pair.
- ``fill_value=0`` replaces missing combinations with zero revenue.

Quick Review

1. What function would you use to read a CSV file into a DataFrame?
2. How can you quickly view the first 10 rows of a DataFrame?
3. Explain the difference between `.loc` and `.iloc`.
4. Write a one liner to filter a DataFrame `df` for rows where column `A` is greater than 5.
5. What does `.groupby('category').sum()` return?

Further Reading

- **Data Cleaning** – `.dropna()`, `.fillna()`, `.replace()`, handling duplicates.
- **Merging & Joining** – `.pd.merge()`, `.concat()`, `.join()`.
- **Time Series Handling** – resampling, shifting, rolling windows.
- **Advanced Aggregations** – `.agg()` with custom functions, multi index groupby.
- **Performance Tips** – using categorical data, `.categorical` dtype, `.astype('category')`.
- **Exporting Data** – `.to_csv()`, `.to_excel()`, `.to_sql()`.

These topics build on the fundamentals covered here and will deepen your proficiency with Pandas.

